BOSS-KAMPF 1 · Welt 1 · Bearbeitungszeit 60 min · Hilfsmittel: Werkzeugkarten

W1·Boss – Die Gildenhalle: das Auftragsbrett

Erreichbar: 24 BE · Papier-Compiler zu e) · Zusätzlich im Lab: **Online-Boss „Der Reitstall der Heldenschmiede“**
Niveau (KC 2017): Grund- und Leistungskurs – c) und d) (Vererbung, Klassenbeziehungen) nur Leistungskurs (eA)

Eure Spielwelt bekommt eine Gildenhalle. Am großen Auftragsbrett hängen Aufträge, die Abenteurerinnen und Abenteurer annehmen, erledigen und gegen Gold eintauschen können. Euer Studio modelliert das Brett mit den Klassen **Auftrag** und **Auftragsbrett**. Diesmal kennst du die Klassen noch nicht – zeig, dass du dein Wissen aus der Heldenschmiede übertragen kannst!

Hinweise zur Bearbeitung

- Bearbeite alle Teilaufgaben **ohne Gerät** auf dem Antwortbogen am Ende dieses Hefts. Erlaubt sind nur deine Werkzeugkarten.
- Implementierungen schreibst du in **Java**, Klassen- und Objektkarten nach den Abi-Konventionen (Sichtbarkeit, deutsche Typnamen, Konstruktor mit c).
- Zeitbudget wie im Abitur: ca. 2,5 Minuten pro BE. Die BE stehen rechts neben jeder Teilaufgabe.

Papier-Compiler

Deine Lösung zu **Aufgabe e)** tippst du direkt nach dem Boss-Kampf **unverändert** in der Online-IDE ab und startest die Tests im Testrunner. Alle Tests grün beim ersten Versuch: **Papier-Compiler bestanden**. Prüfe in e) und f) besonders die Leer-, Index- und null-Abfragen – das sind die typischen Abi-Fallen.

Zusätzlich im Lab: Online-Boss „Der Reitstall der Heldenschmiede“

Im **Algo Quest Lab** (algoquest.frankjuchim.de) wartet der Online-Boss von Welt 1: 25 Minuten Zeitlimit, Auswertung erst am Ende, der beste Versuch zählt. Die Punkte vergibt das Lab automatisch.

Aufgaben

a) Erläutern Sie anhand der Klasse **Auftrag** (Abbildung 1) den grundsätzlichen Aufbau einer Klasse. Gehen Sie dabei auch auf die Sichtbarkeitsattribute (+ und -) sowie auf die beiden Konstruktoren ein. **[4 BE]**

b) Der Programmausschnitt in Abbildung 2 wird ausgeführt.

Stellen Sie den Zustand nach der Ausführung mithilfe von Objektkarten dar. Berücksichtigen Sie dabei die Variablen brett, a1, a2 und a3 sowie die Felder 0 bis 3 der Reihung auftraege.

Geben Sie an, welchen Wert der Aufruf `brett.getAuftrag(2).istErledigt()` anschließend liefert, und begründen Sie Ihre Angabe. **[4 BE]**

c) Für dringende Aufträge gibt es die Unterklasse **Eilauftrag** (Abbildung 3). Außerdem soll eine Klasse **Abenteurer** ergänzt werden: Ein Abenteurer hat einen Namen und ein Goldkonto. Er kann höchstens einen Auftrag gleichzeitig angenommen haben (anfangs keinen). Er kann einen Auftrag annehmen und den angenommenen Auftrag abschließen; dabei wird der Auftrag erledigt und die Belohnung seinem Goldkonto gutgeschrieben.

Ergänzen Sie das Klassendiagramm aus Abbildung 1 um die Klassen Eilauftrag und Abenteurer einschließlich aller Beziehungen. **[4 BE]**

d) Geben Sie die Ausgabe des Programmausschnitts in Abbildung 4 an.

Erläutern Sie, welche Methode getBelohnung bei den drei Aufrufen jeweils ausgeführt wird. **[3 BE]**

e) Die Methode **leichtesterOffenerAuftrag(): Auftrag** der Klasse Auftragsbrett liefert den noch nicht erledigten Auftrag mit der kleinsten Schwierigkeit. Haben mehrere offene Aufträge dieselbe kleinste Schwierigkeit, wird derjenige mit dem kleinsten Index geliefert. Gibt es keinen offenen Auftrag, wird null geliefert. Das Auftragsbrett und die Aufträge bleiben unverändert.

Implementieren Sie die Methode in Java. **[5 BE]**

f) Die Methode **sucheAuftrag(titel: Zeichenkette): Auftrag** soll den ersten Auftrag des Bretts mit dem übergebenen Titel liefern; gibt es keinen solchen Auftrag, soll null geliefert werden. Die Implementierung in Abbildung 5 enthält drei Fehler.

Analysieren Sie die Implementierung: Identifizieren Sie die drei Fehler, erläutern Sie jeweils deren Auswirkung und geben Sie eine korrigierte Implementierung an. **[4 BE]**

Summe: 24 BE

Material

zu den Aufgabenteilen a) bis c), e) und f)

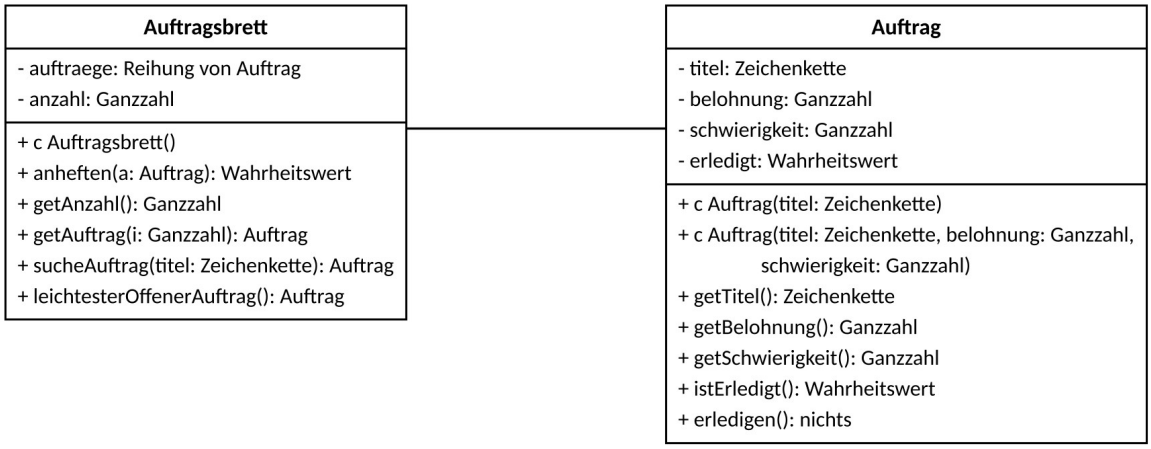


Abbildung 1: Klassendiagramm mit den Klassenkarten der Klassen Auftragsbrett und Auftrag

Erläuterungen zu Abbildung 1

- Der Konstruktor Auftrag(titel) erzeugt einen Standardauftrag mit belohnung = 10 und schwierigkeit = 1.
- Der Konstruktor mit drei Parametern übernimmt die übergebenen Werte. Liegt schwierigkeit nicht im Bereich 1 bis 5, wird schwierigkeit = 1 gesetzt.
- Ein neuer Auftrag ist nicht erledigt. erledigen() setzt erledigt auf wahr.
- Die Reihung auftraege hat die Länge 10. Die angehefteten Aufträge stehen lückenlos in den Feldern 0 bis anzahl − 1; alle übrigen Felder enthalten null.
- anheften(a) legt a im ersten freien Feld ab und liefert wahr. Ist a null oder das Brett voll, wird falsch geliefert.
- getAuftrag(i) liefert den Auftrag mit dem Index i; bei einem ungültigen Index wird null geliefert.

zu Aufgabenteil b)

```
Auftragsbrett brett = new Auftragsbrett();
Auftrag a1 = new Auftrag("Ratten jagen");
Auftrag a2 = new Auftrag("Drachenei bergen", 120, 5);
Auftrag a3 = a1;
brett.anheften(a1);
brett.anheften(a2);
brett.anheften(a3);
a3.erledigen();
a2 = new Auftrag("Kräuter sammeln", 15, 2);
```

Abbildung 2: Programmausschnitt (Hauptprogramm)

zu den Aufgabenteilen c) und d)

```
public class Eilauftrag extends Auftrag {
    private int frist;   // in Tagen

    public Eilauftrag(String titel, int belohnung,
                      int schwierigkeit, int frist) {
        super(titel, belohnung, schwierigkeit);
        this.frist = frist;
    }

    public int getFrist() {
        return frist;
    }

    public int getBelohnung() {
        if (frist <= 2) {
            return super.getBelohnung() * 2;
        }
        return super.getBelohnung() + 10;
    }
}
```

Abbildung 3: Implementierung der Klasse Eilauftrag

zu Aufgabenteil d)

```
Auftrag e1 = new Eilauftrag("Brücke reparieren", 40, 3, 1);
Auftrag e2 = new Eilauftrag("Wolf vertreiben", 40, 3, 5);
Auftrag n1 = new Auftrag("Brief zustellen", 40, 1);
println(e1.getBelohnung() + " " + e2.getBelohnung()
        + " " + n1.getBelohnung());
```

Abbildung 4: Programmausschnitt (Hauptprogramm; println gibt eine Zeile auf der Konsole aus)

zu Aufgabenteil f)

```
public Auftrag sucheAuftrag(String titel) {
    for (int i = 1; i <= anzahl; i++) {
        if (auftraege[i].getTitel() == titel) {
            return auftraege[i];
        }
    }
    return null;
}
```

Abbildung 5: fehlerhafte Implementierung der Methode sucheAuftrag der Klasse Auftragsbrett

Antwortbogen

a) Aufbau der Klasse Auftrag **[4 BE]**

b) Objektkarten nach Abbildung 2 · Wert von brett.getAuftrag(2).istErledigt() **[4 BE]**

Zeichenfläche für die Objektkarten

brett.getAuftrag(2).istErledigt() liefert: _____ Begründung:

c) Ergänzung des Klassendiagramms (Auftrag und Auftragsbrett dürfen Sie verkürzt mit „…“ zeichnen) **[4 BE]**

Zeichenfläche für das Klassendiagramm

d) Ausgabe und Erläuterung **[3 BE]**

Ausgabe: _____

e) Implementierung von leichtesterOffenerAuftrag – Papier-Compiler: sauber und lesbar schreiben! **[5 BE]**

f) Fehleranalyse und korrigierte Implementierung von sucheAuftrag **[4 BE]**

Auswertung (durch die Lehrkraft oder Selbst-/Peer-Korrektur nach Kernpunkten)

Teilaufgabe	a)	b)	c)	d)	e)	f)	Summe
max. BE	4	4	4	3	5	4	24
erreicht							

Korrigiert von: _____ Papier-Compiler e): ☐ grün beim 1. Versuch ☐ grün nach Korrektur ☐ nicht grün